The Prompt Ladder (Week 2)

Name: Bhoompally Kalyan Reddy | Track: Machine Learning / AI Fluency

Baseline: The Embarrassing Start

PROMPT: Write python code to predict traffic.

OUTPUT EXCERPT: import sklearn... df = pd.DataFrame(np.random.randint(0,100,size=(100, 4)))... model = LinearRegression()...

What changed in prompt: **None (Baseline)**

What improved in output: **None**

What still failed / worse: **It used generic dummy data and a Linear Regression model which is useless for my real dataset.**

What to try next: **Add Real Context about the dataset.**

Version 1: + Context

PROMPT: Write python code to predict website traffic drops using DuckDB and a Hugging Face parquet dataset.

*OUTPUT EXCERPT: import duckdb... con.execute("SELECT * FROM hf://datasets...") ... X_train, X_test = train_test_split(df)...*

What changed in prompt: **Added Real Context (+Context).**

What improved in output: **It successfully used the correct DuckDB and Hugging Face libraries.**

What still failed / worse: **It used a standard random train_test_split, which causes massive data leakage between pages on the same website.**

What to try next: **Add Constraints regarding the validation split.**

Version 2: + Constraints

PROMPT: Write python code to predict website traffic drops using DuckDB and a Hugging Face parquet dataset. Constraint: You MUST use GroupShuffleSplit on client_id to prevent data leakage.

OUTPUT EXCERPT: gss = GroupShuffleSplit()... next(gss.split(X, y, groups=df["client_id"]))... print(model.score(X_test, y_test))

What changed in prompt: **Added a strict validation constraint (+Constraints).**

What improved in output: **The data leakage is fixed! The model is now evaluated on unseen clients.**

What still failed / worse: **The output is just a single accuracy number (0.85), which doesn't tell me if it is just guessing the majority class.**

What to try next: **Add a Specified Output Format.**

Version 3: + Output Format

PROMPT: Write python code to predict website traffic drops... You MUST use GroupShuffleSplit. Output Format: Print a clean classification_report table comparing the model to a naive baseline.

OUTPUT EXCERPT: def complex_naive_baseline(row): if row.impressions > 100 and row.clicks < 5... return 1...

What changed in prompt: **Added a required output table (+Format).**

The Prompt Ladder (Week 2)

Name: Bhoompally Kalyan Reddy | Track: Machine Learning / AI Fluency

What improved in output: **It prints a beautiful, easy-to-read metric table comparing the model to a baseline.**
What still failed / worse: **[MADE IT WORSE]: The AI hallucinated a wildly complex baseline rule that makes no sense, ruining the comparison.**
What to try next: **Add Examples of what a good baseline looks like.**

Version 4: + Examples

PROMPT: Write python code to predict website traffic drops... Output Format: Print a classification_report comparing the model to a naive baseline. Example: A naive baseline should simply guess the majority class (e.g., "Not Declining" for every row).

OUTPUT EXCERPT: baseline_preds = np.zeros(len(y_test))... LogisticRegression().fit(X_train, y_train)...

What changed in prompt: **Added a specific example for the baseline (+Examples).**
What improved in output: **The baseline is finally honest and simple.**
What still failed / worse: **The model performance is terrible because it chose Logistic Regression, which cannot handle the non-linear traffic data.**
What to try next: **Add a Clearer Goal specifying the exact model type.**

Version 5: + Clearer Goal

PROMPT: Goal: Train a Random Forest Classifier to predict website traffic drops... [rest of previous prompt]

OUTPUT EXCERPT: RandomForestClassifier(n_estimators=100)... print(classification_report...)

What changed in prompt: **Specified the exact model type (+Clearer Goal).**
What improved in output: **Accuracy jumped significantly because Random Forest handles the non-linear features properly.**
What still failed / worse: **Nothing. This script is ready for production.**
What to try next: **Clean up the prompt into a reusable template for the team.**

Final Reusable Prompt Template

Write a Python script to predict {target_variable} from a {database_type} database.

Context: The dataset is located at {dataset_path}.
Goal: Train a {model_type} to predict the target.
Constraints: You MUST use {validation_strategy} on the {group_column} to prevent data leakage.
Output Format: Output the final evaluation as a clean classification report table comparing the model to a naive baseline.
Examples: A naive baseline should simply guess the majority class (e.g., '0' for every row).